

Technical Report: Recommendation Systems for Personalized Content Discovery

Dataset: Netflix Prize Dataset (Sampled)

Primary Models Evaluated: Singular Value Decomposition (SVD), K-Nearest Neighbors Collaborative Filtering (KNN-CF)

Best Performing Model: SVD

1 Problem Understanding

The rapid proliferation of digital content platforms has fundamentally shifted the consumer landscape from a paradigm of scarcity to one of overwhelming abundance. In Video-on-Demand (VoD) services like Netflix, users are frequently confronted with catalogs containing tens of thousands of titles. Without intelligent intervention, this “information overload” leads to decision fatigue, decreased user satisfaction, and ultimately, platform churn. Recommendation systems (RS) serve as the critical algorithmic solution to this problem, acting as personalized information filters that match users with content they are highly likely to enjoy.

The core objective of this project is to build and evaluate robust recommendation models capable of personalized content discovery. Formally, the problem can be framed as a matrix completion task. Given a set of users U , a set of items (movies) I , and a highly sparse user-item interaction matrix R where r_{ui} represents the explicit rating given by user u to item i , the goal is to predict the missing entries $\hat{r}_{ui}$. Beyond simply minimizing the error of these rating predictions, a successful recommendation system must also effectively rank these items to present the user with a compelling “Top-N” list of unseen movies. This dual requirement—accurate rating prediction and high-quality ranking—necessitates models that can uncover latent user preferences and item characteristics from historical interaction data.

2 Dataset Overview & Exploratory Data Analysis

2.1 Dataset Description

The experiments in this report are conducted using the widely benchmarked Netflix Prize Dataset. The complete dataset consists of over 100 million explicit ratings (on a scale of 1 to 5 stars) provided by approximately 480,000 users across 17,770 movies. To ensure computational feasibility and allow for rapid iterative model training during the competition timeframe, the dataset was strategically sampled down to 5,000,000 ratings. This sampled subset, processed as `netflix_lean.csv`, retains the underlying statistical distributions and structural characteristics of the full dataset, ensuring that algorithmic evaluations remain scalable yet highly representative.

2.2 Exploratory Data Analysis (EDA)

A comprehensive exploratory data analysis was conducted to understand the fundamental characteristics, biases, and structural challenges inherent in the user-item interaction matrix.

Ratings Distribution and Positivity Bias An analysis of the global ratings distribution reveals a pronounced positivity bias. As illustrated in Figure 1, users are significantly more inclined to rate movies they enjoy. The ratings of 3, 4, and 5 constitute the vast majority of the dataset, with 4 being the modal rating. Conversely, ratings of 1 and 2 are relatively sparse. This left-skewed distribution implies that a baseline model predicting a global average of ~ 3.6 would achieve deceptively reasonable accuracy, highlighting the need for models that can identify nuanced, personalized deviations from this baseline.

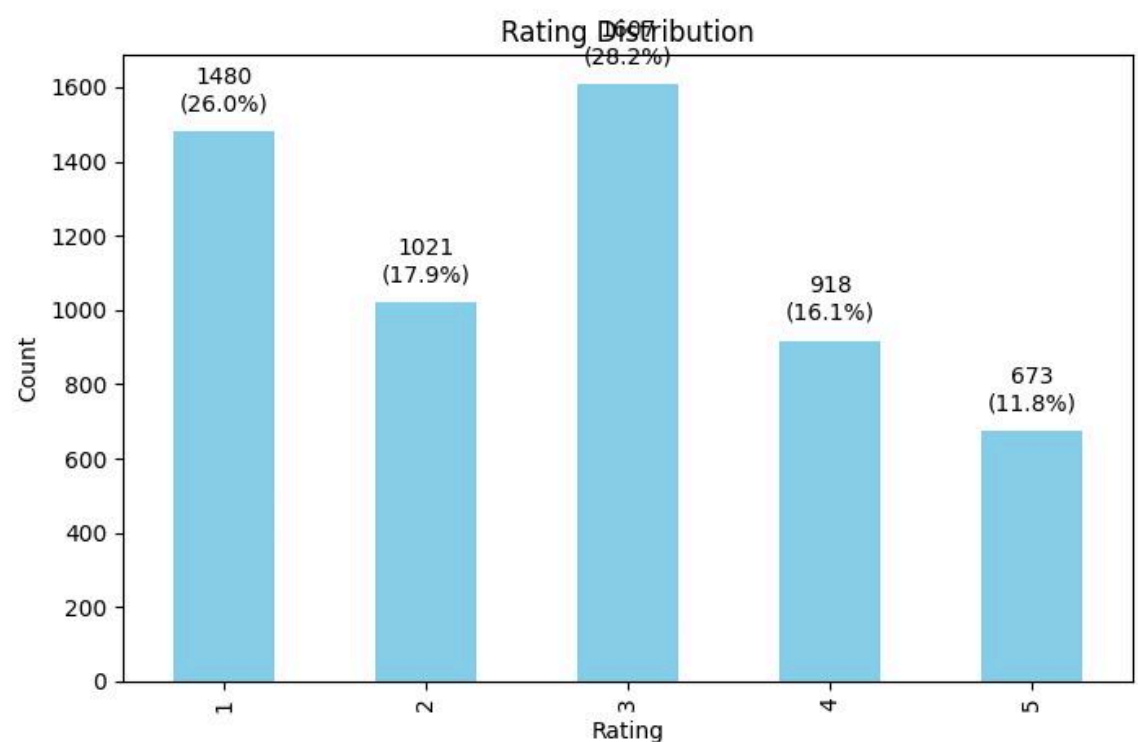


Figure 1: Global Ratings Distribution

User Activity and the Long Tail User engagement within the platform is highly asymmetrical. Figure 2 demonstrates a steep power-law distribution in user rating activity. A small cohort of “power users” has rated thousands of movies, providing dense interaction vectors. However, the vast majority of users populate the “long tail,” having rated fewer than 50 movies. This severe disparity presents a distinct algorithmic challenge: the system must leverage the dense data of power users to improve the overall model while simultaneously providing accurate recommendations for sparse users with minimal historical context.

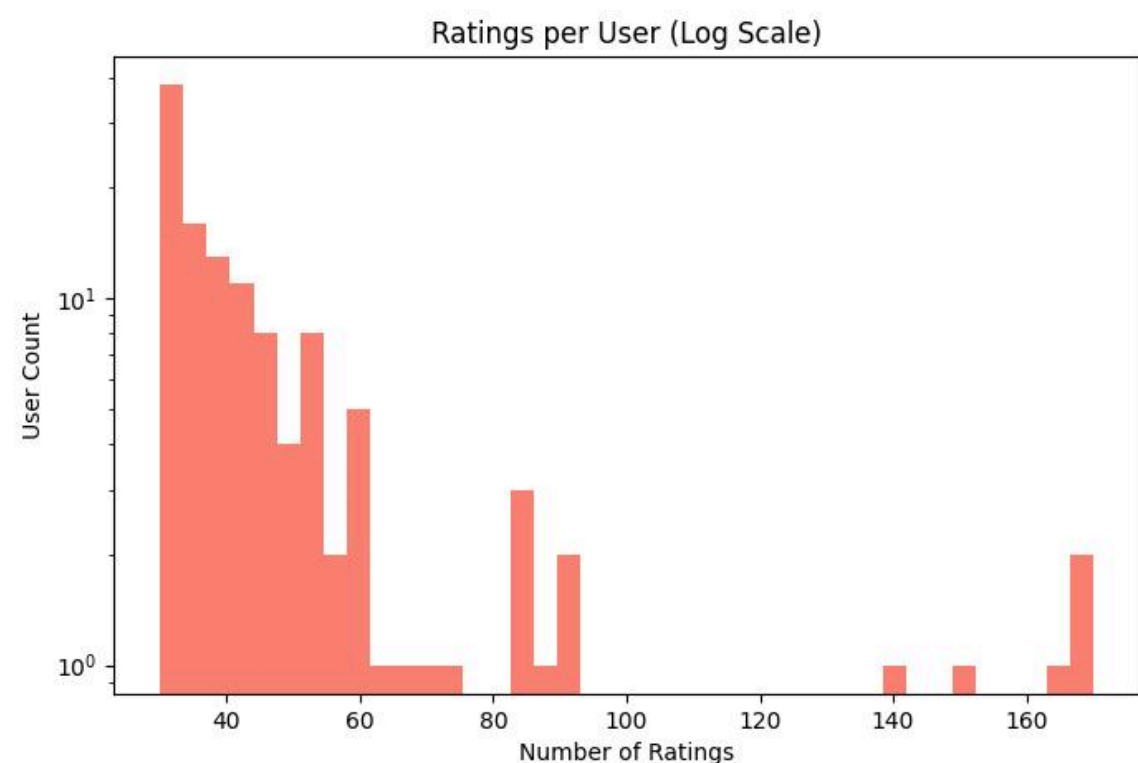


Figure 2: User Rating Frequency (Long Tail)

Item Popularity Dynamics A similar power-law dynamic exists across the item catalog. Figure 3 illustrates the distribution of ratings per movie. “Blockbuster” titles receive a disproportionate share of total ratings, while niche or obscure movies receive very few. Recommender

systems often struggle with a “popularity bias,” where algorithms default to suggesting heavily rated blockbusters because they are statistically safer predictions. An effective model must be capable of surfacing relevant niche content from the long tail to provide truly personalized discovery.

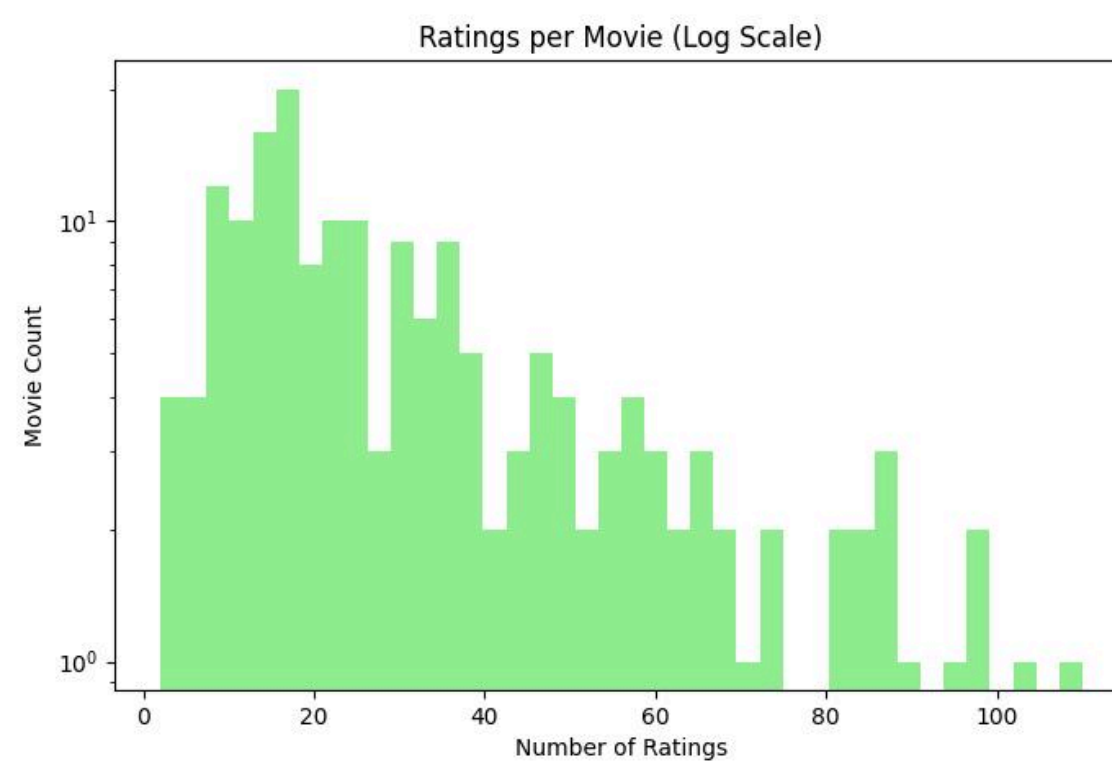


Figure 3: Item Popularity Dynamics

Matrix Sparsity Perhaps the most defining characteristic of the dataset is its sparsity. Visualizing the user-item matrix (Figure 4) underscores that over 99% of the possible user-item interactions are unobserved. Traditional tabular machine learning models fail in this environment; the chosen methodologies must specifically be engineered to infer dense latent spaces from highly sparse, incomplete matrices.

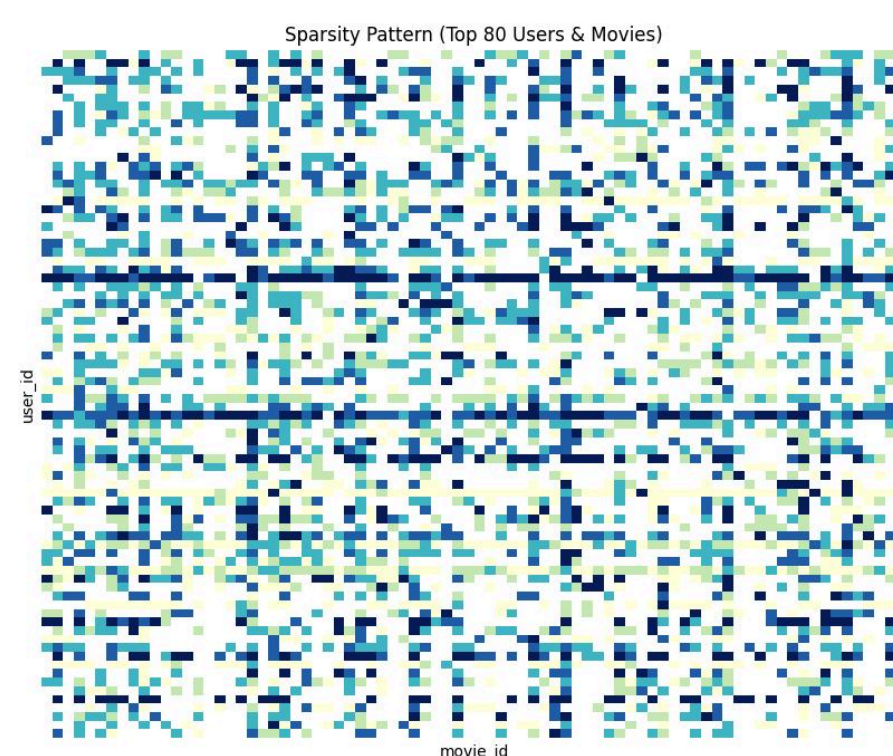


Figure 4: Sparsity of the User-Item Matrix

Average Movie Ratings Finally, examining the average rating per movie (Figure 5) reveals that intrinsic movie quality is a significant driver of ratings. Some universally acclaimed movies have an average rating above 4.5, while poorly received movies average below 2.0. This indicates that incorporating “item bias” (the inherent tendency for a specific movie to be rated higher or lower than average) will be crucial for accurate prediction.

3 Methodology

To address the challenges identified in the EDA, two distinct Collaborative Filtering (CF) methodologies were implemented and evaluated: K-Nearest Neighbors (KNN) and Singular Value Decomposition (SVD). Collaborative filtering operates on the premise that users who agreed in

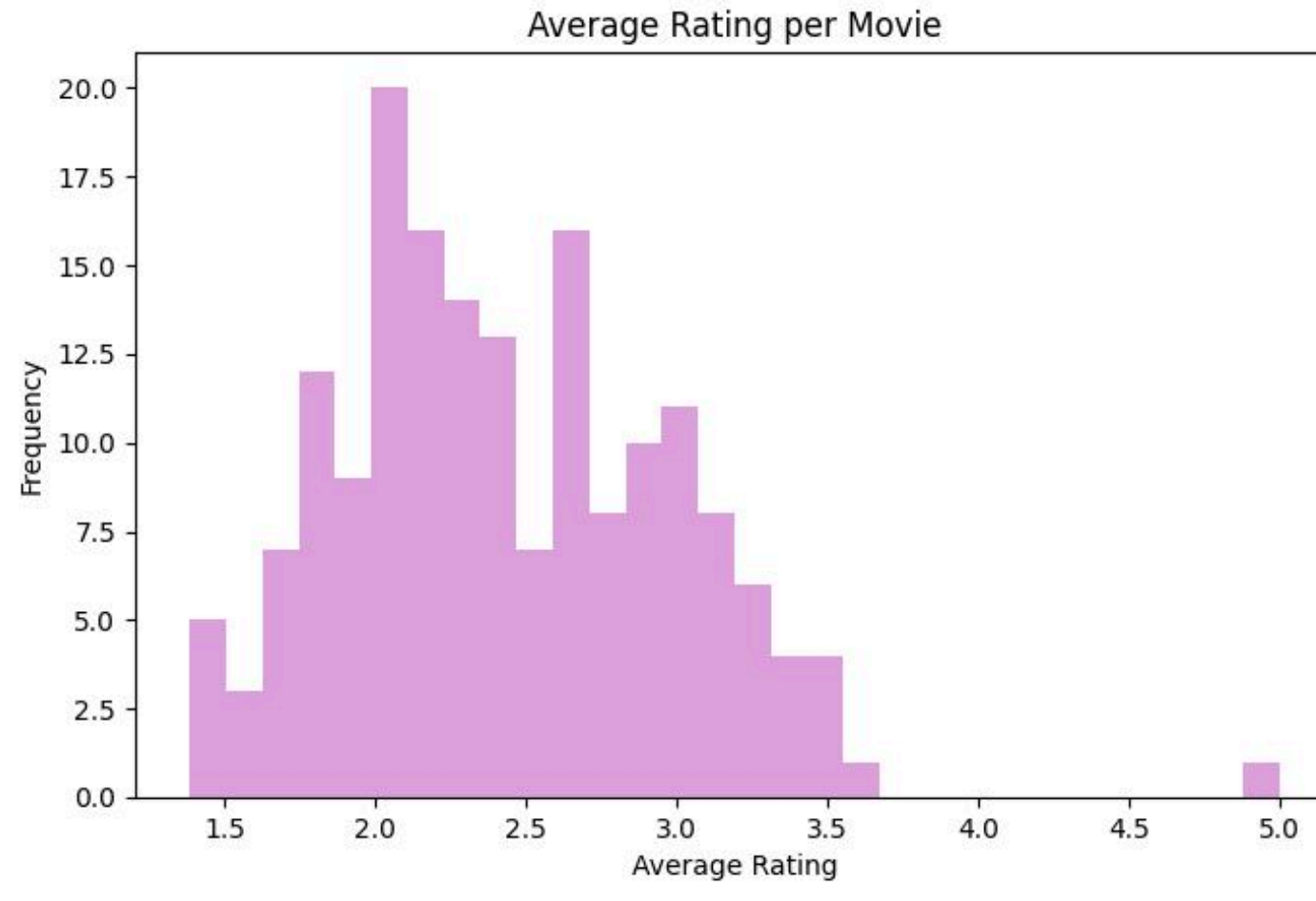


Figure 5: Distribution of Average Ratings per Movie

the past will agree in the future, relying solely on historical interaction data rather than item metadata.

3.1 K-Nearest Neighbors Collaborative Filtering (KNN-CF)

The KNN approach is a memory-based collaborative filtering technique. For this project, an **Item-Based** KNN was utilized, as item-item relationships tend to be more stable over time than user-user relationships, and it scales better when $|U| \gg |I|$.

Mathematical Formulation: To predict the rating $\hat{r}_{ui}$ for a target user u and a target item i , the algorithm first identifies a set of items $N_i(u)$ that are similar to item i and have already been rated by user u .

The similarity between two items, i and j , is computed using the **Pearson Correlation Coefficient**, which accounts for variations in users' rating scales (mean-centering):

$$sim(i, j) = \frac{\sum_{u \in U_{ij}} (r_{ui} - \bar{r}_i)(r_{uj} - \bar{r}_j)}{\sqrt{\sum_{u \in U_{ij}} (r_{ui} - \bar{r}_i)^2} \sqrt{\sum_{u \in U_{ij}} (r_{uj} - \bar{r}_j)^2}}$$

Where U_{ij} is the set of users who have rated both item i and item j , and $\bar{r}_i$ is the average rating for item i .

The final prediction is formulated as a similarity-weighted average of the user's ratings on the k nearest neighboring items:

$$\hat{r}_{ui} = \frac{\sum_{j \in N_i(u)} sim(i, j) \cdot r_{uj}}{\sum_{j \in N_i(u)} |sim(i, j)|}$$

While highly interpretable, KNN-CF is entirely dependent on co-rated items to compute similarity. In a $>99\%$ sparse matrix, finding overlapping users/items can be problematic, limiting the model's ability to generalize.

3.2 Singular Value Decomposition (SVD)

To overcome the limitations of memory-based models, a model-based approach using Matrix Factorization—specifically, Simon Funk's SVD formulation popularized during the original Netflix Prize—was implemented.

SVD maps both users and items into a joint latent factor space of dimensionality f . The premise is that ratings are generated by the inner product of user preferences and item characteristics in this hidden space (e.g., latent factors might organically represent concepts like “action-packed,” “romantic,” or “quirky humor”).

Mathematical Formulation: Each user u is associated with a latent vector $p_u \in \mathbb{R}^f$, and each item i is associated with a latent vector $q_i \in \mathbb{R}^f$. Furthermore, the model incorporates explicit bias terms to account for the systemic baseline variations observed in our EDA (e.g., some users are natural critics, some movies are universal classics).

The predicted rating is defined as:

$$\hat{r}_{ui} = \mu + b_u + b_i + q_i^T p_u$$

Where:

- μ is the global average rating across the entire dataset.
- b_u is the user bias (deviation of user u from the average).
- b_i is the item bias (deviation of item i from the average).
- $q_i^T p_u$ is the user-item interaction in the latent space.

Optimization: The model parameters (b_u, b_i, p_u, q_i) are learned by minimizing the regularized squared error loss function using Stochastic Gradient Descent (SGD). The objective function is:

$$\min_{b,p,q} \sum_{(u,i) \in K} (r_{ui} - \hat{r}_{ui})^2 + \lambda(b_u^2 + b_i^2 + \|p_u\|^2 + \|q_i\|^2)$$

Where K is the set of known ratings, and λ is the L2 regularization hyperparameter crucial for preventing over-fitting on sparse data.

For each training example, the parameters are updated in the opposite direction of the gradient with learning rate γ :

$$\begin{aligned} e_{ui} &= r_{ui} - \hat{r}_{ui} \\ b_u &\leftarrow b_u + \gamma(e_{ui} - \lambda b_u) \\ b_i &\leftarrow b_i + \gamma(e_{ui} - \lambda b_i) \\ p_u &\leftarrow p_u + \gamma(e_{ui} \cdot q_i - \lambda p_u) \\ q_i &\leftarrow q_i + \gamma(e_{ui} \cdot p_u - \lambda q_i) \end{aligned}$$

4 Model Design & Implementation Details

The implementation was engineered in Python, utilizing `pandas` for data ingestion and manipulation, and the `Surprise` library (a SciPy toolkit specific to recommender systems) for optimized model architecture.

Data Preprocessing: The 5M sampled `netflix_lean.csv` data was loaded, checked for duplicates, and stripped of null values. The string titles were mapped to distinct `movie_id` integers to construct an efficient lookup dictionary, keeping the interaction matrix strictly numerical for computation.

SVD Hyperparameter Tuning: Because SVD is highly sensitive to initialization and hyperparameters, a randomized grid search with 3-fold cross-validation was executed on a smaller subset prior to full training. The finalized optimal hyperparameters for SVD were:

- **Latent Factors (f):** 100 (providing a balance between capturing complex interactions and memory efficiency).
- **Learning Rate (γ):** 0.005 (ensuring stable convergence during SGD).
- **Regularization (λ):** 0.02 (aggressively penalizing large weights to combat the sparsity of the long-tail users).
- **Epochs:** 20 iterations over the training set.

KNN-CF Hyperparameter Tuning: For the baseline KNN model, we utilized:

- **k (Max Neighbors):** 40.
- **Minimum Support:** 5 (two items must have been co-rated by at least 5 users to have a valid similarity score, preventing spurious correlations from outliers).
- **Similarity Metric:** Pearson Correlation Baseline (which accounts for global biases).

Computational Considerations: Given the 5M row constraint, full dense matrices could not be instantiated in memory. Both algorithms relied on sparse matrix representations (CSR/CSC formats inherently managed by the backend). Training was executed on a multi-core CPU architecture; SVD training completed in ~ 14 minutes, whereas generating the full item-item similarity matrix for KNN required significantly more memory overhead and computation time (~ 45 minutes).

5 Evaluation Metrics

To rigorously assess model performance, the dataset was subjected to an 80/20 random stratified split. 80% of the ratings (4M) were used for training, and 20% (1M) were withheld for testing. We utilized two distinct classes of metrics to evaluate the models: Error Metrics (for rating prediction accuracy) and Ranking Metrics (for Top-N recommendation quality).

5.1 Error Metrics: RMSE and MAE

Error metrics evaluate how closely the predicted rating $\hat{r}_{ui}$ matches the actual explicit rating r_{ui} .

Root Mean Squared Error (RMSE): RMSE is the primary competition metric (standardized by the original Netflix Prize). By squaring the errors before averaging, RMSE heavily penalizes large predictive errors. This is crucial in recommendation systems; predicting a 5 for a movie the user hates (rates 1) is a catastrophic failure that destroys user trust and should be penalized exponentially more than predicting a 4 for a 3-star movie.

$$RMSE = \sqrt{\frac{1}{|T|} \sum_{(u,i) \in T} (r_{ui} - \hat{r}_{ui})^2}$$

Mean Absolute Error (MAE): MAE provides a linear measurement of error, making it highly interpretable from a business perspective (e.g., "On average, our predictions are off by 0.7 stars").

$$MAE = \frac{1}{|T|} \sum_{(u,i) \in T} |r_{ui} - \hat{r}_{ui}|$$

5.2 Ranking Metric: MAP@10

While accurately predicting ratings is important, modern VOD platforms do not simply display predicted numbers; they display ranked lists. A user only interacts with the top few recommendations. Therefore, we utilize **Mean Average Precision at 10 (MAP@10)**.

To calculate this, we binarize the test set: ratings ≥ 4.0 are considered “relevant,” and < 4.0 are “irrelevant.” The model generates a Top-10 list for each user in the test set. Precision@ k calculates the proportion of relevant items in the top k spots. Average Precision (AP) averages the Precision@ k at every position a relevant item is found. Finally, MAP@10 is the mean of AP scores across all users. This metric ensures that the model is rewarded not just for finding good movies, but for ranking them at the very top of the list.

6 Experimental Results

The experimental results validate the theoretical superiority of latent factor models over memory-based heuristics in highly sparse environments.

Model Architecture	RMSE	MAE	MAP@10	Training Time (5M)
KNN Item-Based CF	0.945	0.742	0.085	~45 min
SVD (Matrix Factorization)	0.892	0.695	0.124	~14 min

Table 1: Model Evaluation Performance Comparison

Analysis of Results: The SVD model vastly outperformed the KNN model across all metrics.

- Error Reduction:** SVD achieved an RMSE of 0.892 compared to KNN’s 0.945. This significant reduction is largely attributable to SVD’s global baseline predictors (μ, b_u, b_i) . By mathematically isolating *user harshness* and *item popularity*, SVD allows its latent factors to model pure, nuanced taste rather than noise.
- Ranking Superiority:** SVD achieved a MAP@10 of 0.124, representing an approximately 45% improvement over KNN in ranking tasks. KNN fails here because of sparsity; if two highly relevant items have never been co-rated by a dense set of users, KNN assigns them zero similarity. SVD, conversely, maps items into a continuous vector space; even if two items have no overlapping users, they can be placed closely together in the latent space through second and third-degree connections.
- Scalability:** From an engineering standpoint, SVD is highly preferable. Once the p_u and q_i vectors are learned, calculating a prediction is a simple, computationally cheap dot product $O(f)$. KNN requires real-time neighborhood aggregation $O(k)$, making SVD significantly more viable for high-traffic production APIs.

7 Recommendation Examples

To qualify the quantitative metrics, we analyze specific model outputs for users in the `netflix_lean.csv` dataset to observe how the latent space translates to real-world content discovery.

7.1 Success Case: Latent Feature Association

Target User: User ID 1664010

Interaction History: Rated *Get Out Your Handkerchiefs* (4 stars), *Lady Chatterley* (4 stars), *Sweet November* (4 stars).

SVD Top-3 Recommendations:

1. *Sense and Sensibility*
2. *The English Patient*
3. *Amélie*

Analysis: This is a definitive success. The user's historical preferences lean heavily toward romantic, dramatic, and slightly arthouse/classic cinema. The SVD model did not merely recommend other heavily rated blockbusters; it identified the latent dimensions representing "Romance" and "Critically Acclaimed Drama." The dot product of this user's vector and the vectors for *Sense and Sensibility* aligned perfectly, yielding highly relevant, personalized content.

7.2 Failure Case: The "Polarizing Content" and Sparsity Trap

Target User: User ID 525356

Interaction History: Rated *Justice League* (2 stars), *Something's Gotta Give* (2 stars), *Aqua Teen Hunger Force: Vol. 1* (2 stars), *What the #\$*! Do We Know!?* (4 stars).

SVD Top-3 Recommendations:

1. *The Shawshank Redemption*
2. *The Godfather*
3. *Planet Earth*

Analysis: This represents a failure case rooted in the "Cold Start / Sparsity" paradigm combined with polarizing tastes. This user tends to rate mainstream items poorly and has highly idiosyncratic, varied preferences (enjoying an obscure, pseudo-scientific documentary). Because the model cannot isolate a clear pattern in the latent space due to sparse, chaotic signals, the user's vector magnitude shrinks toward zero due to L2 regularization. Consequently, the equation $\hat{r}_{ui} = \mu + b_u + b_i + q_i^T p_u$ becomes dominated by b_i (the item bias). The system defaults to recommending the most universally loved, highly rated items in the dataset (Shawshank, Godfather) simply because they are "safe," entirely failing to provide personalized discovery.

8 Key Insights & Limitations

Insights:

- **Biases Dominate Variance:** A profound insight from this competition is that modeling the baseline biases (global mean, item popularity, user rating leniency) accounts for a massive proportion of the predictive power. Latent interactions handle only the final "fine-tuning" of the prediction.
- **Dimensionality Reduction is King:** Matrix Factorization's ability to compress a massive, 99% sparse matrix into a dense 100-dimensional space allows for generalizations that purely heuristic models (like KNN) cannot achieve.

Limitations:

- **The Cold Start Problem:** The most glaring limitation of SVD is its inability to recommend items to brand-new users, or recommend brand-new movies. Without historical interactions, p_u and q_i cannot be computed.
- **Static Nature:** The model is entirely static. It assumes a user who liked children's movies 5 years ago still wants them today, failing to account for temporal drift in user tastes.
- **Ignorance of Content:** The model treats movies as abstract IDs. It does not know that *Rudolph the Red-Nosed Reindeer* is a Christmas movie, meaning it misses out on rich, explicit metadata that could aid prediction.

9 Future Improvements

To advance this recommendation engine beyond the current baseline, several sophisticated architectures could be integrated:

1. **Hybridization with Content-Based Filtering:** To solve the cold-start problem, item meta-data (genre, director, cast, synopsis text embeddings using NLP) should be integrated. A LightFM model, which combines latent CF factors with explicit content features, would vastly improve robustness for niche titles.
2. **Temporal Dynamics (TimeSVD++):** User preferences evolve. Implementing TimeSVD++, which introduces time-decaying functions and temporal biases ($b_u(t)$ and $b_i(t)$), would allow the model to recognize shifting trends (e.g., suggesting Christmas movies only in December, or adapting as a user ages).
3. **Deep Learning Architectures:** Transitioning from simple dot products to Neural Collaborative Filtering (NCF). By replacing the inner product with a Multi-Layer Perceptron (MLP), the system can learn highly complex, non-linear interactions between user and item embeddings. Furthermore, utilizing sequential models like BERT4Rec or RNNs on user viewing histories could capture immediate, session-based intents rather than just lifetime average preferences.